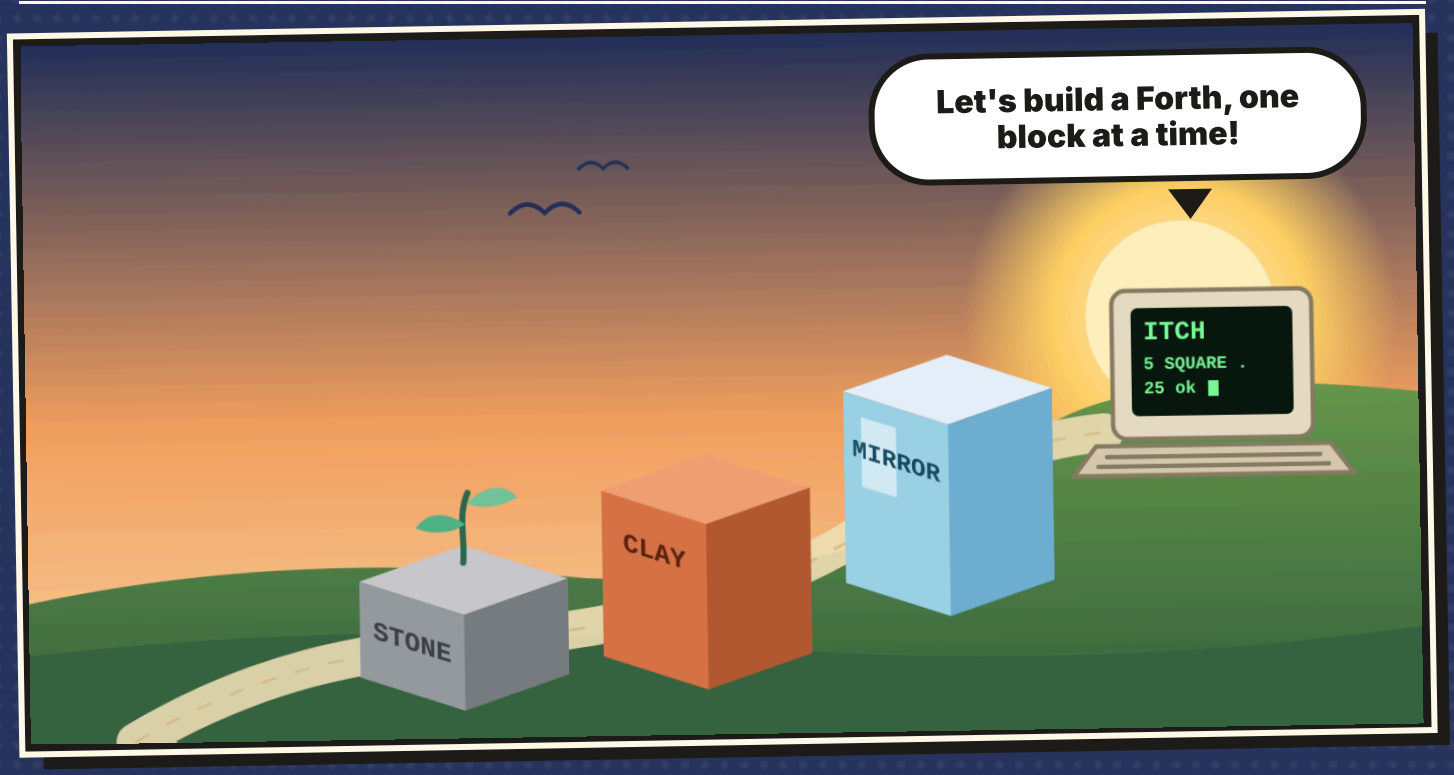
FORTH IN STONE

A FORTH SPECIAL · OCTOBER 2026 · UPPSALA, SWEDEN



The good building blocks of Forth

PART 1
STONE
Forth in JavaScript: run words, read text

PART 2
CLAY
the compiler, written in Forth

PART 3
MIRROR
Forth in Forth: it reads itself

PART 4
ITCH
the real machine, in 64K

PART 5
DEEP DIVE
words that make word-makers

PART 6
FIELD GUIDE
run now, or build later?

PART 7
LITTLE LANGUAGES
Forth as a DSL in a DSL

PART 8
YOUR TURN
try it yourself, with hints

POW!
TYPE IT IN AND WATCH IT GROW!

JIMMY SÖDERMAN
Uppsala, Sweden · October 2026

PROGRAMMING · A FORTH SPECIAL

Build a Forth in three steps

Forth is the language that builds itself. In three small steps you watch it happen: first a stone that only runs words, then clay that learns to make them, then a mirror where Forth reads Forth.

BY JIMMY SÖDERMAN · UPPSALA, SWEDEN

Most languages arrive finished. A compiler you cannot see turns your text into a program, and that is that. Forth is different. Forth is made of words, and the words that compile your program are words too: you can read them, change them and add new ones. That is why a whole Forth can grow from a very small seed.

Words and the stack

You type words separated by spaces, and Forth runs them one after another. Numbers wait on a pile called the **stack**. `2 3 +` puts 2 and 3 on the pile; `+` takes both and leaves 5; `.` takes the top number and prints it.

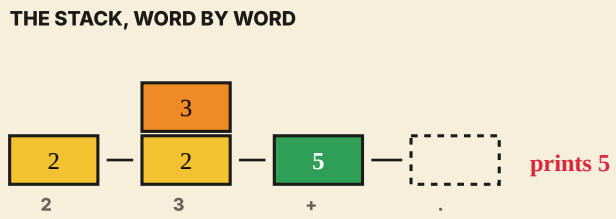


FIGURE 1 *Each word changes the stack. The newest number sits on top.*

New words from old

You make new words from old ones. `: SQUARE DUP * ;` makes SQUARE: copy the top number, multiply. Now `5 SQUARE .` prints 25. Every word lives in the **dictionary**, a list Forth searches, and a new word is at once as good as any built-in one.

Forth has almost no fixed syntax. IF, loops, comments, even the colon itself are words. So you can grow the language in any direction, including its own compiler. This article does exactly that, in three steps that share one small piece of JavaScript.

HOW TO FOLLOW ALONG

Open **forth-in-stone.html** in any browser. Three buttons load the three steps; RUN runs the text in the box. Change anything and run again. Each step brings the ones before it, so every step works on its own.

Stone, clay and mirror

The **stone** is the part that cannot change: about 74 lines of JavaScript, the same in every step. The **clay** is Forth, which the stone lets you shape. The **mirror** is the moment Forth sees itself: it writes its own reader and takes over the reading. After the three steps comes ITCH, the real machine this road leads to.

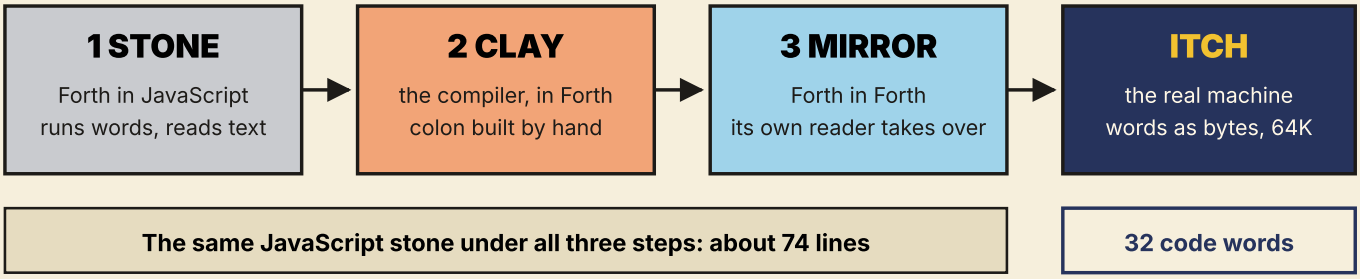
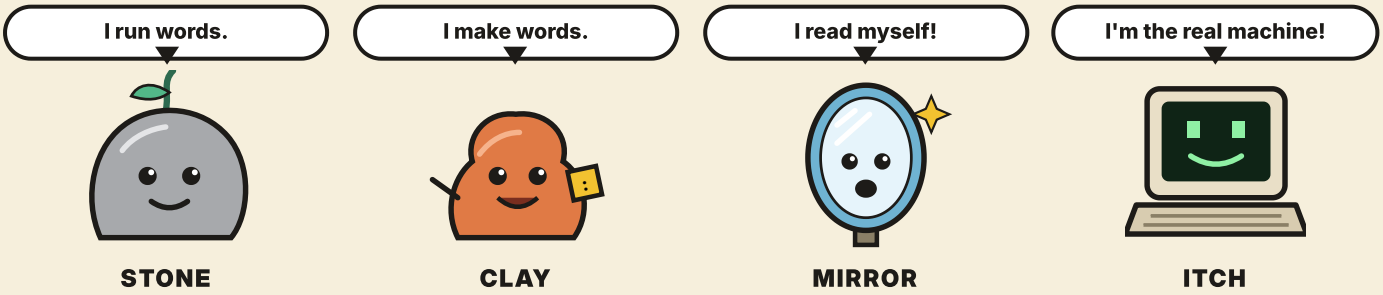
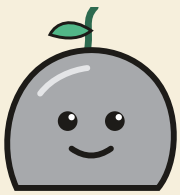


FIGURE 2 *The ladder. Each step adds Forth, never JavaScript.*



STONE: Forth in JavaScript



I can run any word you give me. Make new ones? **Not yet!**

WHAT HAPPENS

JavaScript reads your text, word by word. A word runs; a number goes on the stack. That's all.

WHAT'S SPECIAL

There is no colon, so Forth can't make new words yet. But both interpreters are already here: the outer one reads text, the inner one runs a word's body.

In step 1 Forth can run words, but it cannot make new ones. There is no colon and no semicolon. What there is, is the stone: the JavaScript at the bottom of the page. It holds two interpreters, and telling them apart is the first lesson of Forth.

The inner interpreter runs words

A word made of other words has a **body**: a list of words. To run it, the inner interpreter walks that list one cell at a time. The place it has reached is `ip`, the list it walks is `th`. Each step takes the next word and runs its code. This loop is called NEXT; on a real machine it is a handful of instructions.

LISTING 1 · THE INNER INTERPRETER

```
const EXIT = { code() {      // back out
  [th, ip] = R.pop(); } };
const LIT = { code() {       // a number
  S.push(th[ip++]); } };
const docol = w => {         // step in
  R.push([th, ip]); th = w.body; ip = 0; };
function execute(w) {
  const base = R.length; w.code(w);
  while (R.length > base) {  // NEXT
    const x = th[ip++]; x.code(x); } }
```

Three tiny pieces do all the work. `docol` steps into a word: it saves where we were on the return stack R and starts on the word's body. `EXIT` steps back out. `LIT` says "the next cell is a number, not a word".

CONNECTION

Keep LIT in mind. In Part 2, `LITERAL` builds exactly this pair of cells, LIT and a number, into a new word.

The outer interpreter reads text

The outer interpreter is the one you talk to. It takes the next token from the text, looks it up in the dictionary and runs it. If the token is not a word, it tries to read it as a number and puts that on the stack. If that fails too, you get the token back with a question mark.

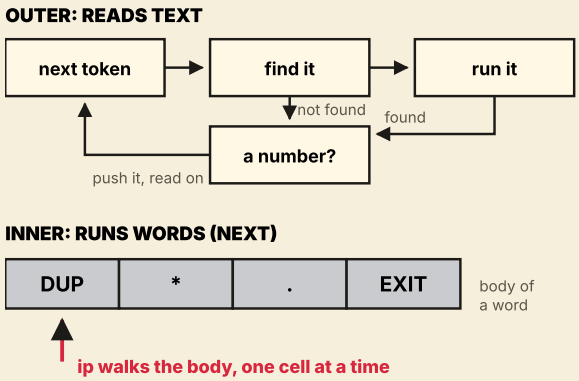


FIGURE 3 *Two interpreters. The outer one reads your text; the inner one walks a word's body.*

The reader at work

The outer interpreter has only three cases, and every Forth you will ever meet has the same three. Watch it read `2` `DUP` `FOO`:

1 IT READS A NUMBER

```
2
```

The reader looks for a word called 2. There is none, so it reads it as a number and pushes it.

stack: 2

2 IT READS A WORD

```
DUP
```

DUP is in the dictionary, so the reader runs it. DUP copies the top of the stack.

stack: 2 2

3 IT READS NONSENSE

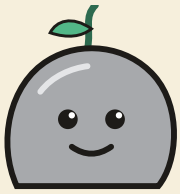
```
FOO
```

Not a word, not a number. The reader stops and gives the token back with a question mark.

FOO ?

CONNECTION

Remember these three cases. In Part 3, Forth writes them again itself, as INTERPRET.



Words, numbers, memory.
But no colon. Yet!

The primitives

Besides the two interpreters, the stone gives 35 primitives: the only words written in JavaScript. Everything else in this article is built from them.

GROUP	WORDS
Stack, maths	`DUP DROP SWAP + - * = OR`
Memory	`@ ! , STATE`
Output	`. CR`
Reader's parts	`WORD ' FIND NUMBER? NOT-FOUND UNQUOTE EXECUTE`
Builder's parts	`HEADER REVEAL IMMEDIATE COMPILE, LITERAL HERE PATCH BRANCH 0BRANCH`
Strong words' parts	`(CREATE) >BODY (DOES>)`
Threads	`EXIT LIT`

A calculator with a memory

With no colon, step 1 is a calculator. But it already has memory, and a word is a value you can hold:

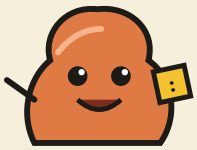
LISTING 2 · STEP 1 SESSION

```
2 3 + .  10 4 - .  6 7 * .    \ 5 6 42
1 2 SWAP . .  5 DUP * .       \ 1 2 25
WORD X (CREATE) 0 ,           \ a variable
7 X !  X @ .                  \ 7
' DUP .                       \ DUP
```

`WORD X` reads a name, `(CREATE)` makes a word that gives an address, and `0 ,` puts one empty cell there. That is all a variable is: a name and a cell.

NEXT...
CLAY BUILDS A COLON, STARTING FROM FIVE CELLS!

CLAY: the compiler, in Forth



No colon? **No problem!** I'll build one by hand, cell by cell.

WHAT HAPPENS

Forth builds a colon by hand from a few cells, then writes its whole compiler in Forth: IF, loops, comments, and four strong words.

WHAT'S SPECIAL

Compile time: some words run while the compiler reads. And not one line of JavaScript is added.

In step 2 Forth learns to make new words, and every line of that is Forth. Not one line of JavaScript is added. The key idea is **compile time**: some words run while the compiler is reading, instead of being built in. That is the missing half that turns a calculator into a language that can teach its own reader new tricks.

First, the colon by hand

There is no `:` yet, so the first three words are put together cell by cell from the builder's parts, the way you would do it on bare metal:

LISTING 3 ·] : ; BUILT BY HAND

```
WORD ] HEADER  -1 LITERAL ' STATE COMPILE,
  ' ! COMPILE,  ' EXIT COMPILE,  REVEAL
WORD : HEADER  ' WORD COMPILE, ' HEADER COMPILE,
  ' ] COMPILE,  ' EXIT COMPILE,  REVEAL
WORD ; HEADER  ' EXIT LITERAL ' COMPILE, COMPILE,
  ' REVEAL COMPILE,  0 LITERAL ' STATE COMPILE,
  ' ! COMPILE,  ' EXIT COMPILE,  REVEAL IMMEDIATE
```

Read the first line slowly. WORD reads the name `]`. HEADER starts a new, still hidden word. LITERAL and COMPILE, add its cells. REVEAL makes it findable. That is all a compiler is: words writing cells into a list.

THE BODY OF] , CELL BY CELL



Five cells: "compile from now on".

FIGURE 4 *What the first line of Listing 3 builds.*

`:` reads a name, starts a header and calls `]`. `;` adds EXIT, reveals the word and sets STATE back to 0. It is marked IMMEDIATE, so it runs at once when the compiler meets it. From here on, Forth has a real colon.

STATE: TWO MOODS

STATE is the switch of the outer interpreter. At 0 it runs each word it reads. At -1 it builds each word into the new word instead. An IMMEDIATE word always runs, whatever the mood.

Jumps that fill their own holes

Control flow is just IMMEDIATE words that write jumps. IF writes a conditional jump with an empty hole for the target, and leaves the hole's position on the stack. THEN, later, fills the hole with the place it has reached.

: ZERO? 0 = IF 1 ELSE 2 THEN . ;

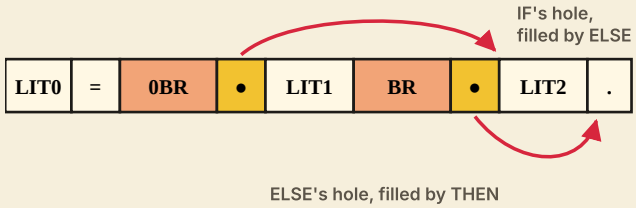


FIGURE 5 *Each ● is a hole left by IF or ELSE and filled later with a jump target.*

LISTING 4 · CONTROL FLOW, IN FORTH

```
: [    0 STATE ! ; IMMEDIATE
: [']  ' LITERAL ; IMMEDIATE
: IF     ['] 0BRANCH COMPILE, HERE 0 COMPILE,
         ; IMMEDIATE
: THEN   HERE SWAP PATCH ; IMMEDIATE
: ELSE   ['] BRANCH COMPILE, HERE 0 COMPILE,
         SWAP  HERE SWAP PATCH ; IMMEDIATE
: BEGIN  HERE ; IMMEDIATE
: UNTIL  ['] 0BRANCH COMPILE, COMPILE, ; IMMEDIATE
: WHILE  ['] 0BRANCH COMPILE, HERE 0 COMPILE,
         SWAP ; IMMEDIATE
: REPEAT ['] BRANCH COMPILE, COMPILE,
         HERE SWAP PATCH ; IMMEDIATE
: ?DUP   DUP IF DUP THEN ;
```

The four strong words

Four words make Forth different from almost every other language: **CREATE**, **DOES>**, **DEFER** and **IS**. None of them is in the stone. Forth builds all four in Forth, from a few small parts the stone leaves open. They are worth reading slowly, because each one does something surprising.

1. CREATE: a word with a home for data

CREATE reads a name and makes a new word. The new word owns a place in memory, and when it runs, all it does is push that place's address. What you put there is up to you: `CREATE X 0 ,` makes X with one cell holding 0. That is a variable.

> **CONNECTION**
> `(CREATE)` is the part you used by hand in Part 1 to make X. CREATE only adds WORD in front.

LISTING 5 · CREATE, AND WHAT IT MAKES

```
: CREATE    WORD (CREATE) ;  \ read a name, make
it
: VARIABLE  CREATE 0 , ;
VARIABLE X   7 X !   X @ .   \ 7
```

So far nothing strange. A variable is just a CREATEd word with one cell. The surprise comes when you give such a word an *action*.

2. DOES>: one definition, two lives

DOES> splits a definition in two. The part before it runs **once**, when a new word is made. The part after it runs **every time** that new word runs. One definition writes the behaviour of every word it makes.

LISTING 6 · DOES>

```
: DOES>    ['] (DOES>) COMPILE, ; IMMEDIATE
: CONSTANT  CREATE , DOES> @ ;
```

DOES> is IMMEDIATE, so it runs while CONSTANT is being compiled. It builds in one cell, `(DOES>)`, and the compiler carries on. CONSTANT's body becomes: `CREATE , (DOES>) @ EXIT`. Follow it through the three moments below.

1 YOU DEFINE IT

```
: CONSTANT
  CREATE , DOES> @ ;
```

DOES> runs now, at compile time, and builds in `(DOES>)`. CONSTANT's body is: **CREATE , (DOES>) | @ EXIT**

2 YOU MAKE A WORD

```
42 CONSTANT ANSWER
```

CONSTANT runs its first half: CREATE makes ANSWER, `,` stores 42 in its cell. Then `(DOES>)` hands ANSWER the rest of the body, **@ EXIT**, as its action, and CONSTANT stops.

3 THE WORD RUNS

```
ANSWER .
```

ANSWER pushes its address, then runs its action: @ fetches what is stored there.

42

The part after DOES> was written inside CONSTANT, but it never runs for CONSTANT. It runs for ANSWER, and for every other constant you make. That is a word that makes words *and* writes how they behave.

HOW (DOES>) DOES IT, IN THE STONE

It remembers where it is in CONSTANT's body: the very next cell. It makes that place the newest word's action, then leaves CONSTANT early, just as EXIT would. Three lines of JavaScript, and the rest is Forth.

With CREATE and DOES> you can make any kind of word: arrays, tables, counters, colours. And DEFER, the next strong word, is made from exactly these two.

NEXT...
DEFER AND IS: PLUGS YOU CAN SWAP!

DEFER and IS: plugs you can swap

Usually, when one word calls another, the call is fixed the moment it is compiled. DEFER breaks that rule. A deferred word is a **plug socket**: it holds another word and runs it, and you can change which word is in the socket at any time, even after other words already use it.

3. DEFER: a socket, made from CREATE and DOES>

LISTING 7 · DEFER

```
: DEFER  CREATE 0 ,  DOES> @ EXECUTE ;
```

Read it with the three moments. When you write `DEFER GREET`, the first half runs: CREATE makes GREET, and `0 ,` gives it one empty cell, the socket. When GREET runs, the second half runs: `@` fetches the word in the socket and `EXECUTE` runs it. An empty socket says so: "DEFER not set".

> **CONNECTION**
> DEFER is nothing new: CREATE gives the socket, DOES> gives the action, EXECUTE runs the plug. Part 5 builds the same thing from a plain VARIABLE.

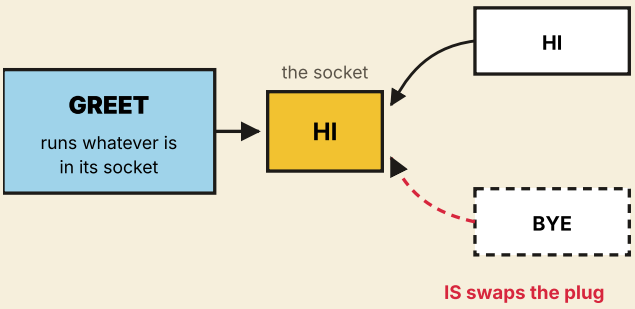


FIGURE 6 *GREET is a socket. IS takes one plug out and puts another in.*

4. IS: put a word in the socket

LISTING 8 · IS AND [IS]

```
: IS     ' >BODY ! ;           \ at the prompt
: [IS]   ' >BODY LITERAL ['] ! COMPILE,
         ; IMMEDIATE            \ inside a word
```

Follow `' HI IS GREET` word by word. `' HI` puts the word HI on the stack. Then IS runs: its `'` reads the next name, GREET; `>BODY` turns GREET into the address of its socket; `!` stores HI there. Now GREET says 1.

Inside a definition it must happen later, when the word runs, so there is `[IS]`. It is IMMEDIATE: while compiling, it finds GREET's socket now and builds in "store into this socket" for later. That is the same split ITCH uses for every word like this.

LISTING 9 · PLUGS AT WORK

```
DEFER GREET
: HI 1 . ;    : BYE 2 . ;
: FLIP ['] BYE [IS] GREET ;
' HI IS GREET  GREET      \ 1
FLIP           GREET      \ 2
```

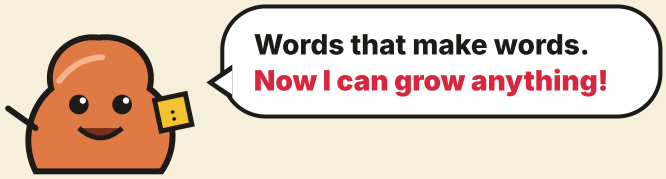
> **WHY THIS IS SO STRONG**
> Any word that calls GREET will do something new after a swap, without being compiled again. In step 3, this is how Forth makes even its own reader changeable.

The compiler at work

LISTING 10 · ALL OF STEP 2 IN USE

```
: SQUARE ( n -- n*n ) DUP * ;
5 SQUARE .                          \ 25
42 CONSTANT ANSWER  ANSWER .        \ 42
: ZERO? ( n -- )  0 = IF 1 ELSE 2 THEN . ;
0 ZERO?  5 ZERO?                    \ 1 2
: COUNTDOWN ( n -- )
  BEGIN DUP . 1 - DUP 0 = UNTIL DROP ;
5 COUNTDOWN                         \ 5 4 3 2 1
```

By the end of step 2, Forth has built its own compiler from about 25 lines of Forth: brackets, tick, seven control-flow words, strings, comments and the four strong words.



NEXT...
MIRROR READS ITSELF!

Part 3 · Step 3

MIRROR: Forth in Forth



WHAT HAPPENS

Forth writes its own reader, INTERPRET, and runs it. From that line on, every word is read by Forth, and the JavaScript reader just waits.

WHAT'S SPECIAL

The reader is now a Forth word, so you can change how Forth reads, from Forth. UNKNOWN is a DEFER: a socket in the reader itself.

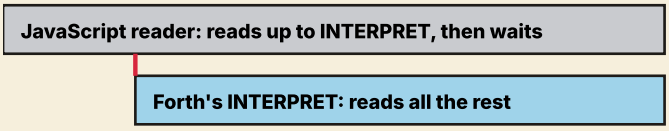
In step 3 Forth writes its own outer interpreter and hands the reading over to it. This is the moment the language comes alive. A language that reads itself like this is called metacircular, and Forth gets there in about ten lines.

Three parts first. `FIND` leaves the word and 1 if it is IMMEDIATE, the word and -1 if not, or the token and 0 if it is unknown. `NUMBER?` leaves the number and -1, or the token and 0. `?DUP` copies the top only if it is not 0.

LISTING 11 · FORTH'S OWN READER

```
DEFER UNKNOWN  ' NOT-FOUND IS UNKNOWN
: INTERPRET ( -- )
  BEGIN WORD ?DUP WHILE
    FIND ?DUP IF
      1 = STATE @ 0 = OR
      IF EXECUTE ELSE COMPILE, THEN
    ELSE NUMBER? IF STATE @ IF LITERAL THEN
         ELSE UNKNOWN THEN
    THEN
  REPEAT ;
INTERPRET
```

It does what the stone's reader does: take a token, find it, then run it or build it in, or read it as a number. The difference is the last line. When `INTERPRET` runs, it reads the rest of the text itself, and the JavaScript reader just waits for it to finish.



From the line INTERPRET on, every word is read by Forth.

FIGURE 7 *The handover.*

To prove it, change the reader from Forth. UNKNOWN is a plug that decides what happens to a word Forth does not know:

CONNECTION

INTERPRET is Part 1's three cases, written in Forth. UNKNOWN is a DEFER from Part 2. Every part of the climb is used here.

LISTING 12 · CHANGE THE READER

```
: SHRUG ( token -- ) DROP 0 . ;
' SHRUG IS UNKNOWN
HELLO WORLD     \ 0 0, not an error
```

STILL AS FREE AS A FORTH WITH NO STONE

The stone is still underneath, but everything above it, the reader, the compiler and all the syntax, is Forth and can be changed from Forth. Every real Forth works this way, ITCH included: a few code words at the bottom, Forth all the way up.

NEXT...
ITCH: THE REAL MACHINE!

Next stop: ITCH



Forth in Stone shows how Forth works. ITCH shows how it runs on a real machine: a complete 1985-style Forth system on the same ideas. The big change is the body. In Stone, words are JavaScript objects; Forth can run them but cannot look inside. In ITCH everything is bytes in one 64K memory, so Forth can read, change and save its own body.

ITCH'S 64K: ONE MEMORY FOR EVERYTHING

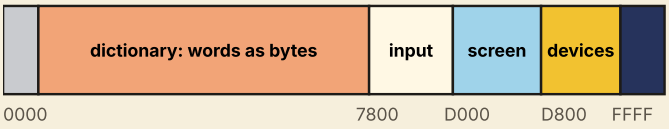


FIGURE 8 *A simplified map of ITCH's memory. Every word, every pixel and every device register is an address.*

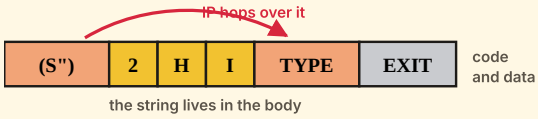
	FORTH IN STONE	ITCH
Words live as	JavaScript objects	Bytes in one 64K memory
See inside a word?	No	Yes: SEE shows any word
Compiling	One reader with STATE	Two loops, no STATE
Host-language words	35 primitives	32 code words a 6502 could run
Saves itself	No	As a ROM, no seed needed
Devices	Text output	Screen, sprites, keys, mouse, sound, clock
Many things at once	One program	Tasks and motors on the beat
Catching mistakes	Unknown words, empty stack, empty DEFER	Crossed IF/THEN, name clashes, stack checks

Stone does not ship POSTPONE, which lets a compiling word build in another word; ITCH has it, and Part 6 writes it for Stone in three lines of Forth. Everything else carries over: both interpreters, IMMEDIATE words, CREATE and DOES>, DEFER and IS, and a reader written in Forth.

DATA AND CODE, SIDE BY SIDE

Why did 1980s home computers love indirect threading? Because IP, the inner interpreter's pointer, can read **data** as well as code. Most words leave IP alone, but a few read the cells after them: LIT takes one number, and ITCH's `(S")` takes a whole string.

: GREET S" HI" TYPE ;



LISTING 13 · (S") IN ITCH

```
: (S")  ( -- a n )  R> COUNT 2DUP + >R ;
```

Read it: `R>` takes the return address, which points right at the string. `COUNT` gives its address and length. `2DUP +` is the place just after the text, and `>R` makes that the new return address, so NEXT carries on after the string. No jump is ever compiled.

To be fair: subroutine-threaded and native Forths can keep strings inline too, by reading their return address: the same `R>` trick. Indirect threading just makes it natural and compact, which mattered in a 64K machine.

CONNECTION

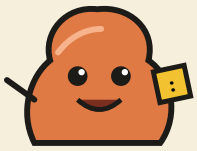
LIT from Part 1 is the one-cell version of this trick: it reads the next cell, then moves IP past it.

The rest is just memory layout.

NEXT...

DEEP DIVES: THE STRONG WORDS, AND RUN NOW OR BUILD LATER!

Words that make words that make words



Can a word that makes words make more word-makers? Yes!

In Part 2, CONSTANT made words. Now we go one level deeper: a word that makes word-makers. It only needs a second CREATE and a second DOES> inside the first action. Every example here was run in Forth in Stone. Type them after step 3, but delete the SHRUG line first, or a typo just prints 0.

Nesting: MAKER, FAMILY and KID

LISTING 14 · TWO DOES> IN ONE WORD

```
: MAKER ( n "name" -- )
  CREATE ,                  \ life 1: when MAKER
runs
  DOES> CREATE @ ,          \ life 2: when FAMILY
runs
  DOES> @ 100 + ;           \ life 3: when KID runs
7 MAKER FAMILY
FAMILY KID
KID .                       \ 107
```

Each DOES> splits off one more life. The rule from Part 2 still holds: **DOES> hands the rest of the body to the newest CREATEd word.** Follow it:

1 DEFINE MAKER

Both DOES> run now and build in two `(DOES>)` cells. MAKER's body has three parts.

2 7 MAKER FAMILY

Life 1 runs: CREATE makes FAMILY and stores 7. The first `(DOES>)` gives FAMILY the rest: lives 2 and 3.

3 FAMILY KID

Life 2 runs: CREATE makes KID and copies FAMILY's 7 into it. The second `(DOES>)` gives KID life 3.

4 KID .

Life 3 runs: fetch KID's 7, add 100.

107



Three generations from one definition.

FIGURE 9 *A family tree of words.*

The first life makes n empty cells. The second life is just `+`: the index you give, plus the array's address, is the address of that cell. In Stone a cell is one step of memory; in ITCH it is two bytes, so ITCH's version says `CELLS +`.

CONNECTION

This is the same move as CONSTANT: the first life sets up memory, the second life decides what the name does. Arrays, tables, colours and counters all follow this pattern.

LOOK OUT

- The second DOES> acts on **KID**, not on FAMILY: it is the newest CREATEd word when life 2 runs.
- Inside life 2, @ fetches FAMILY's own data. FAMILY's address is on the stack because a DOES> word always starts by pushing it.

An array, in three lines

LISTING 15 · ARRAY

```
: ARRAY ( n "name" -- )
  CREATE BEGIN DUP WHILE 0 , 1 - REPEAT DROP
  DOES> + ;                 \ index + address
5 ARRAY A
42 3 A !   3 A @ .          \ 42
```

More ideas to try

- A table of colour names that prints a name for a number.
- A word-maker for 2D points that keeps x and y.
- A counter that adds one each time it runs.

Clever plugs: DEFER and IS

A DEFER is a socket: a word whose action you can change later. That one idea solves three old problems in programming. All three run in Forth in Stone.

1. Using a word before it exists

A word can only call words that already exist. So how do two words call each other? Make one of them a socket first, and plug it in later.

LISTING 16 · EACH CALLS THE OTHER

```
DEFER ODD?
: EVEN? ( n -- f )
  DUP 0 = IF DROP -1 ELSE 1 - ODD? THEN ;
: ODD?-REAL ( n -- f )
  DUP 0 = IF DROP 0 ELSE 1 - EVEN? THEN ;
' ODD?-REAL IS ODD?
4 EVEN? .   7 EVEN? .   7 ODD? .   \ -1 0 -1
```

2. Hooks: change a word that is already in use

Every word that calls a socket sees the new plug at once, without being compiled again. Part 3's UNKNOWN is a hook like this: the reader calls it, and you decide what it does. A program can offer hooks for drawing, for errors or for logging, and let its users plug in their own words.

3. A plain VARIABLE can do it too

A DEFER is a cell holding a word. A VARIABLE is a cell. So you can build a socket by hand:

LISTING 17 · A HOME-MADE SOCKET

```
VARIABLE ACTION
: HI 1 . ;   : BYE 2 . ;
' HI  ACTION !   ACTION @ EXECUTE   \ 1
' BYE ACTION !   ACTION @ EXECUTE   \ 2
```

It works, but you must write `ACTION @ EXECUTE` each time. DEFER packs those three words into the name itself, and IS packs `' >BODY !` into one word. That is why DEFER is the better way.

> **CONNECTION**
> This is Listing 7 again, unpacked: CREATE 0 , is the VARIABLE, DOES> @ EXECUTE is what you just typed by hand.

Another way: a flag instead of a name

Some small Forths do IS differently: their IS (here IS!) only raises a flag, and the deferred word itself checks the flag. If it is up, the word stores the plug instead of running.

LISTING 18 · THE FLAG TRICK

```
VARIABLE IS?
: IS!  -1 IS? ! ;
: DEFER2  CREATE 0 ,
  DOES> IS? @ IF 0 IS? ! ! ELSE @ EXECUTE THEN ;
DEFER2 GREET2        \ BYE from Listing 17
' BYE IS! GREET2   GREET2      \ 2
```

LOOK OUT

- **The flag trick has hidden state.** IS! must be followed straight away by the deferred word. Anything that runs a DEFER2 in between takes the plug by mistake.
- **"DEFER not set":** a socket is empty until you plug something in. Plug it in before anything calls it.
- **Same stack picture:** a plug must take and leave the same things on the stack as the socket promises, or every caller breaks.

THE BEST WAY

- Use DEFER and IS: the name says what it is, and IS reads the target's name, so nothing is hidden.
- Inside a definition use `[IS]`, the split ITCH uses for every word like it.

NEXT...
RUN NOW, OR BUILD LATER? A FIELD GUIDE!

Run now, or build later?



Every Forth word answers one question: do I run now, or build later?

Most confusion in Forth comes from one thing: words that run at different times. Here they all are in one place. Learn the three times first, and the rest falls into place.

The three times

1 · DEFINE THE TOOL
`: SQUARED POSTPONE DUP POSTPONE * ; IMMEDIATE`

2 · USE THE TOOL TO BUILD
`: T2 5 SQUARED . ;` SQUARED runs: builds DUP *

3 · RUN WHAT WAS BUILT
`T2` DUP * runs, and . prints 25

FIGURE 10 *POSTPONE acts at time 1, SQUARED at time 2, DUP and * at time 3. (POSTPONE is written in Listing 20.)*

WORD	WHEN	WHAT IT DOES
`'`	now	reads the next name, gives the word
`[']`	while building	the same, built in as a value
`EXECUTE`	now	runs a word held on the stack
`COMPILE,`	now	builds a word into the new word
`LITERAL`	now	builds a number (LIT n) into the new word; not IMMEDIATE in Stone, so inside a word it acts later, as in UNLESS
`IMMEDIATE`	once	makes the newest word run while building
`[`	while building	STATE off: run the next words now
`]`	now	STATE on: build again
`[COMPILE]`	while building	builds in the next word, even an IMMEDIATE one
`POSTPONE`	while building	makes this word build the next word later
`INTERPRET`	now	the reader loop (Part 3)
`EVALUATE`	now	reads a string as if it were typed (ITCH)

EXECUTE or COMPILE,: the same word, two choices

Put a word on the stack with `'`. Now you can run it at once with EXECUTE, or build it into the new word with COMPILE,. That is exactly the choice INTERPRET makes for every word it reads, using STATE.

CONNECTION
Look at Listing 11: `IF EXECUTE ELSE COMPILE, THEN`. The whole reader is this one choice.

' or [']?

' reads a name **when it runs**. Inside a definition that is too late: it reads the next word of whatever comes after.

LISTING 19 · THE TICK TRAP

```
: BAD ' ;          \ ' runs when BAD runs
BAD DUP .          \ prints DUP: BAD ate it
: GOOD ['] DUP ; \ DUP is chosen now
```

POSTPONE and [COMPILE]

Stone does not ship them, so here they are, in Forth:

LISTING 20 · TWO TOOLS FOR TOOL-MAKERS

```
: [COMPILE] ' COMPILE, ; IMMEDIATE
: POSTPONE  WORD FIND 1 = IF COMPILE,
  ELSE LITERAL ['] COMPILE, COMPILE, THEN
  ; IMMEDIATE
```

[COMPILE] builds in the next word now, even an IMMEDIATE one. POSTPONE looks at the word first. An IMMEDIATE word is built in as it is, so it runs later, when the new tool is used. A normal word gets three cells instead: LIT, the word, and COMPILE,, so the tool builds it in when it is used.

LISTING 21 · NEW CONTROL WORDS

```
: MY-IF  POSTPONE IF ; IMMEDIATE
: UNLESS 0 LITERAL POSTPONE = POSTPONE IF
  ; IMMEDIATE
: T  0 UNLESS 77 . THEN ;    T      \ 77
: T4 1 MY-IF 9 . THEN ;     T4     \ 9
```

Variables that choose, and what to look out for

A variable can decide whether a word runs now or builds later. STATE is the famous one, but it is not the only way, and it has a catch.

STATE-smart words

Stone's `S"` asks STATE what to do: `STATE @ IF LITERAL THEN`. At the prompt it leaves the string; inside a definition it builds the string in. One word, two behaviours, chosen by a variable.

CONNECTION
STATE, from Part 2, is just a variable: cell 0 of the memory. It is how Stone's reader chooses between running and building.

THE CATCH

- A STATE-smart word looks at STATE **when it runs**, not when you meant. Tick it and EXECUTE it later, or POSTPONE it into a tool, and it may pick the wrong behaviour.
- That is why ITCH has no STATE at all. It splits such words in two, one for each side: IS and [IS], TO and [TO].

CONNECTION
Part 2 already met this split: IS at the prompt, [IS] inside a word. Now you know why.

EVALUATE: a string becomes source

ITCH can read a string as if you typed it. It saves where the reader is, points it at the string, runs INTERPRET, and puts everything back. Because the old place waits on the return stack, EVALUATE can nest.

LISTING 22 · EVALUATE, IN ITCH

```
: EVALUATE  ( a n -- )
  SRC @ >R  #TIB @ >R  >IN @ >R   \ save
  #TIB ! SRC !  0 >IN !  INTERPRET \ read
  R> >IN !  R> #TIB !  R> SRC ! ; \ restore
```

Stone cannot do this: its input is a list of tokens cut up in JavaScript, not text in memory. In ITCH the input is bytes like everything else, so Forth can point the reader anywhere.

CONNECTION
This is Part 3's INTERPRET with three cells saved around it. Mirror plus memory equals EVALUATE.

WHAT TO LOOK OUT FOR

- **Tick inside a word:** use ['] . Plain ' reads the next name when the word runs.
- **Forgetting IMMEDIATE:** a tool like MY-IF without it is built in instead of run, and nothing happens until much later.
- **Holes left open:** every IF needs its THEN. An open hole leaves a number on the stack while building; ITCH checks for this.
- **Crossed control:** `BEGIN … IF … UNTIL … THEN` mixes up the holes. ITCH tags every hole to catch it.
- **The newest word:** IMMEDIATE and DOES> act on the newest word. Make another word in between, and they act on that one.
- **STATE-smart words** pick their behaviour late. Prefer split pairs.

THE BEST WAYS

- Use POSTPONE when writing tools. It handles normal and IMMEDIATE words, so you need neither COMPILE, tricks nor [COMPILE].
- Use ' at the prompt, ['] inside definitions.
- Prefer DEFER to a variable plus EXECUTE, and IS to flag tricks.
- Keep one word for each side instead of STATE-smart words, as ITCH does.



Run now or build later: now you always know which!

Forth as a DSL in a DSL



Every reader is a little language. **And you can write your own!**

WHAT HAPPENS

A reader is just a loop of words. Write a new loop, or teach the old one new kinds of tokens, and you have a new little language inside Forth.

WHAT'S SPECIAL

There is no fixed grammar to break out of. Each little language (a DSL) is built from Forth words, and can hold its own little languages in turn.

An old Forth idea says that every interpreter and every compiler in Forth is a small language of its own. You have already met two: Part 1's reader, and Part 3's INTERPRET. ITCH takes the idea further, in four ways.

1. Two loops, one table

ITCH has no STATE. It has two reading loops that are the same except for one cell:

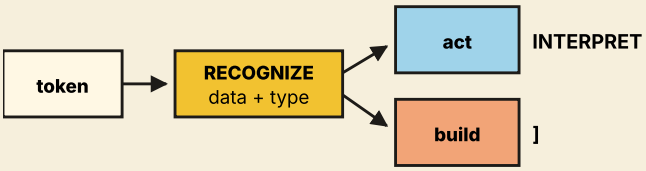
LISTING 23 · ITCH'S TWO LOOPS

```
: INTERPRET  BEGIN BL WORD DUP C@ WHILE
   RECOGNIZE @ EXECUTE REPEAT DROP ;
: ]  BEGIN BL WORD DUP C@ WHILE
   RECOGNIZE 2 + @ EXECUTE REPEAT DROP ;
```

RECOGNIZE turns a token into some data and a **type**. A type is a pair of words: what to do when acting, and what to do when building. INTERPRET runs the first, `]` runs the second.

LISTING 24 · TWO TYPES

```
\              act          build
CREATE T-WORD  ' EXECUTE ,  ' INLINE, ,
CREATE T-NUM   ' NOOP ,     ' LIT, ,
```



One table, two little languages: acting and building.

FIGURE 11 *Every token gets a type, and the loop picks a column.*

CONNECTION

This is Part 6's question, run now or build later, answered by a table instead of a variable. It is why ITCH needs no STATE.

2. Recognizers: new kinds of tokens

RECOGNIZE asks a list of **receptors**, newest first. Each one takes the token and either says "mine", with data and a type, or passes it on with 0. Add a receptor with `+REC`, and the reader understands a new kind of token everywhere, at the prompt and inside words.

LISTING 25 · CHARACTERS LIKE 'A'

```
: REC-CHAR  ( c-addr -- c T-NUM | c-addr 0 )
  DUP C@ 3 = IF DUP 1+ C@ 39 = OVER 3 + C@ 39 =
  AND IF 2 + C@ T-NUM EXIT THEN THEN 0 ;
' REC-CHAR +REC   'A' .                  \ 65
```

A receptor can also just watch. SENSE prints every token in brackets and passes it on, so you see the reader at work:

LISTING 26 · A SENSOR

```
: SENSE  ( c-addr -- c-addr 0 )
  91 EMIT DUP COUNT TYPE 93 EMIT 0 ;
' SENSE +REC  3 4 + .  -REC   \ [3][4][+][.]7
```

3. Word groups: each language its own words

A little language needs its own vocabulary. In ITCH a VOCABULARY is itself a receptor: expressed, its words are found; silenced, they are not. MEMBRANE, SURFACE and SEAL hide helper words, so only the language's own words show.

LISTING 27 · GROUPS AND PRIVATE WORDS

```
VOCABULARY GAME  GAME GROWS  : SCORE 100 ;
TRUNK GROWS
GAME EXPRESS  SCORE .  GAME SILENCE    \ 100
MEMBRANE : HALF 2 / ;  SURFACE
: QUARTER HALF HALF ;  SEAL
40 QUARTER .                           \ 10
```

After SEAL, HALF cannot be found any more, but QUARTER still uses it. The language shows QUARTER and keeps its parts to itself.

4. It started in Stone

CONNECTION
Stone's UNKNOWN socket (Part 3) is a one-slot version of a receptor: it decides what happens to a token the reader does not know. ITCH turns that one slot into a whole list.

LOOK OUT

- **Order matters:** the newest receptor is asked first. One that says "mine" too often hides all the others.
- **Take sensors out again** with -REC, or every token is printed forever.
- **A sealed word** cannot be found by name. Seal only helpers.

THE BEST WAY

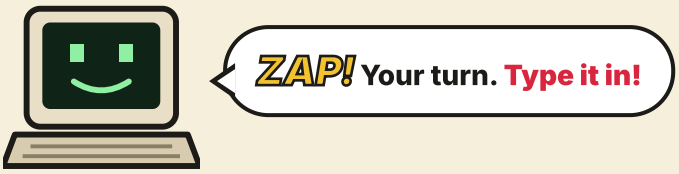
- Grow a little language from small words, one receptor or group at a time, and test each at the prompt.
- Give each receptor one clear kind of token, with one type for acting and building.



A language inside a language inside a language. Turtles all the way down!

Part 8 · Your turn

Try it yourself



The best way to learn Forth is to type it. Open forth-in-stone.html, pick a step, change the text in the box and press RUN. Every exercise below works with the words of its step.

Step 1, STONE

- ☐ Work out (3 + 4) × 5 on the stack, with no brackets.
- ☐ Make two variables by hand, store 6 and 7, and print their product.
- ☐ Print `' SWAP` and `' +`. What does a word look like as a value?

Step 2, CLAY

- ☐ Write `CUBE` from `SQUARE`, and test it on 3.
- ☐ Write `NOT` with IF, ELSE and THEN: 0 becomes -1, anything else becomes 0.
- ☐ Make a defining word `DOUBLED` with CREATE and DOES>, so `21 DOUBLED X X .` prints 42.
- ☐ Build one more word by hand, cell by cell, the way `]` is built.

Step 3, MIRROR

- ☐ Change UNKNOWN so an unknown word is printed back instead of refused.
- ☐ Make INTERPRET print every token it reads, and watch Forth read itself.

Parts 5 to 7

- ☐ Change MAKER so a KID adds 200, and make two families.
- ☐ Write `2ARRAY`, where each item has two cells.
- ☐ Write `UNLESS` with POSTPONE, then use it.
- ☐ Fix `: BAD ' ;` so it always gives DUP.
- ☐ In ITCH: put SENSE in and type a definition. Which tokens does `]` see?
- ☐ In ITCH, a challenge: write a receptor that reads `%101` as a binary number.

Then ITCH

- ☐ Type `SEE CONSTANT` in ITCH and compare it with the one line of CONSTANT in step 2.

HINTS

CUBE: `: CUBE DUP SQUARE * ;`

DOUBLED: `: DOUBLED CREATE , DOES> @ 2 * ;`

NOT: `: NOT 0 = IF -1 ELSE 0 THEN ;` (or simply `0 =`)

2ARRAY: `DOES> SWAP DUP + +` doubles the index.

BAD: `: GOOD ['] DUP ;`

A printing reader: put `DUP .` right after `WORD ?DUP WHILE` in Listing 11.

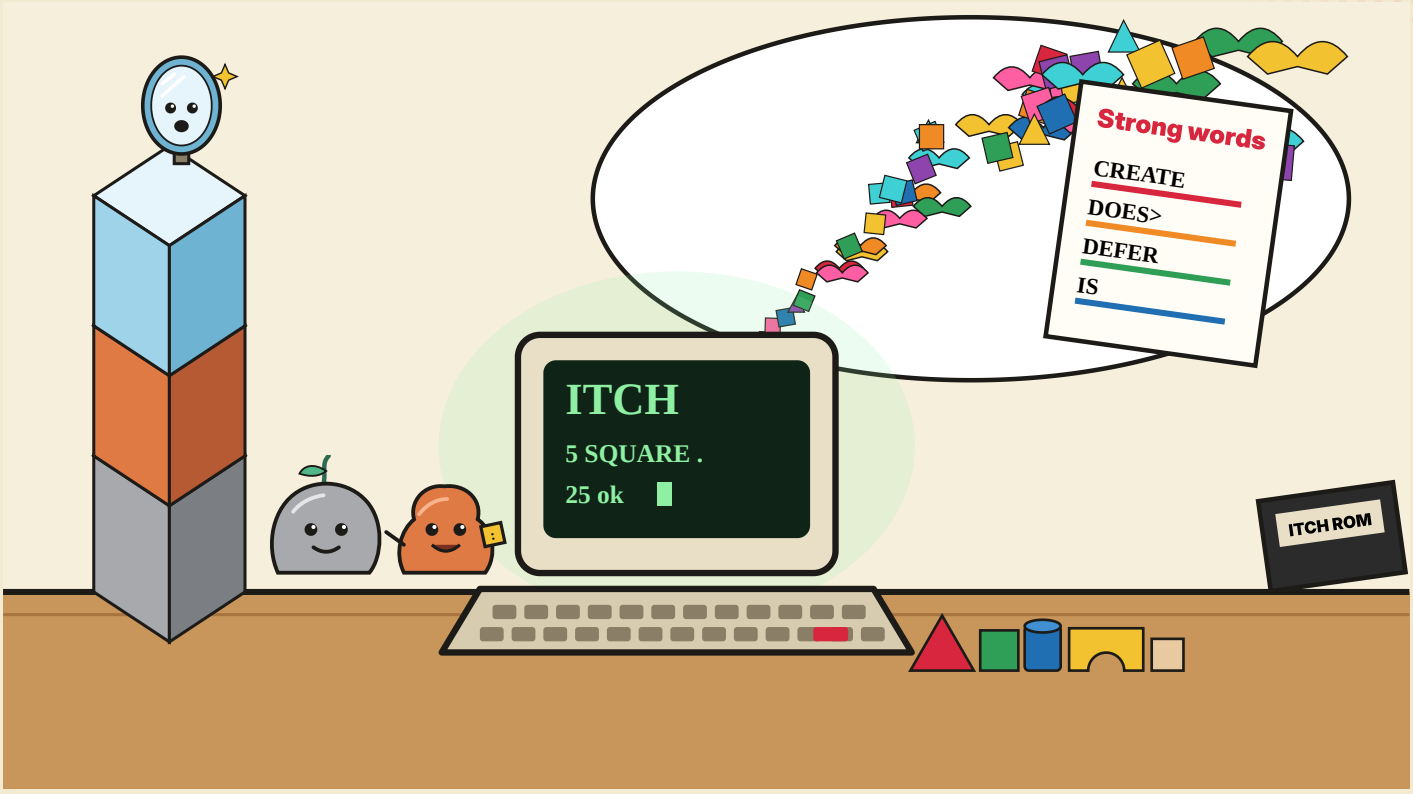
What you have built

A stone of 74 lines that runs words and reads text. A compiler of about 25 lines of Forth, started by hand from five cells of `]`. A reader of ten lines that lets Forth read itself. These are the good building blocks of Forth, and every one of them is waiting for you inside ITCH.

ABOUT THIS ARTICLE

Written by Jimmy Söderman in Uppsala, Sweden, October 2026, as an introduction to Forth for students and a first step on the road to ITCH, a 1985-style Forth system.

BUILD YOUR OWN FORTH!
KIDS & ADULTS ALIKE!



INCLUDES THE STRONG WORDS CARD FOR EXTRA POWER!

Introducing **ITCH**, the 1985-style Forth system you can read from the first byte to the last. Built from the good building blocks: **Stone** runs the words, **Clay** makes them, **Mirror** lets Forth read itself. No magic, no hidden compiler. Just words, all the way up.

Includes:

- **32 code words** a 6502 could run
- **Video chip:** 320×200, tiles and 128 sprites
- **Sound chip:** four voices
- **Tasks and motors** on the beat
- **Save itself** as a ROM, and a disk of demos

ITCH

A Forth for pure creation. Built with Stone, Clay and Mirror.

AVAILABLE WHEREVER FORTH IS TYPED IN!